Team 2
Voting System
Software Design Document

Name (s): Kongmeng Thao, Ly Xiong, Cindy Su, Elsdon Chang
Lab Section: 002
Workstation: CSCI 5801

Date: (02/27/2025)

TABLE OF CONTENTS

1. INTRODUCTION 2

1.1 Purpose 3

1.2 Scope 3

1.3 Overview 3

1.4 Reference Material 3

1.5 Definitions and Acronyms 4

2. SYSTEM OVERVIEW 4

3. SYSTEM ARCHITECTURE 5

3.1 Architectural Design 5

3.2 Decomposition Description 6-7

3.3 Design Rationale 7

4. DATA DESIGN 8

4.1 Data Description 8

4.2 Data Dictionary 9

5. COMPONENT DESIGN 10

6. HUMAN INTERFACE DESIGN 11

6.1 Overview of User Interface 11

6.2 Screen Images 11

6.3 Screen Objects and Actions 11

7. REQUIREMENTS MATRIX 12

8. APPENDICES 12

1. INTRODUCTION

1.1 Purpose

The software design document (SDD) describes the architecture and system design for the voting system, which includes both plurality voting and single transferable voting (STV). The intended audience for this document is software developers, testers, and project stakeholders. This document gives an overview of the system's design, including its components, modules, and interactions, to ensure a clear understanding of the system structure and functionality.

1.2 Scope

The voting system processes election ballots and determines winners based on either the plurality or STV algorithm. The system's goal is to make processing election results more efficient, accurate, and user-friendly for election officials. It can support up to 100,000 ballots, ensuring fair vote counting, and is consistent with the objective of providing a secure and reliable voting system.

1.3 Overview

This document provides a detailed description of the design and development process for the voting system. It's organized into sections that cover the system's purpose, scope, architecture, data design, component design, user interface, and requirements matrix. Each section is organized to make it easier for the user to understand and implement the system.

1.4 Reference Material

IEEE Template for Software Design Documents:

https://canvas.umn.edu/courses/483118/assignments/4495497?module_item_id=14021311

1.5 Definitions and Acronyms

All definitions and acronyms which are used in this document are described in the following table:

Plurality voting	A voting method where the candidate with the most votes wins.
SDD	Software Design Document
Sequence diagram	This diagram shows interactions between actors and the system and between system components.
Single transferable voting	A voting system where voters rank candidates in order of preference. To win, the candidate must reach the Droop Quota.
STV	Single transferable voting
UML	Unified Modeling Language
UML activity diagram	This diagram shows the activities involved in a process or in data processing.
UML class diagram	This diagram shows the object classes in the system and associations between these classes.

2. SYSTEM OVERVIEW

The voting system is a new, self-contained product that handles Plurality voting and STV. The system will take in preprocessed ballot data from CSV files and compute voting results based on the selected algorithm. Plurality voting is where the candidate with the most votes wins and ties are resolved with random selection. STV is a voting system where voters rank candidates in order of preference. To win, the candidate must reach the Droop Quota. The system will prompt the user for the voting method, number of seats, and audit file name if STV is selected in the terminal. If invalid input is entered, the system will display an error message to the user in the terminal and terminate the program. After the system processes the votes successfully, a snapshot of the election results will be printed out in the terminal. The snapshot will include winners, losers, percentage of votes received if plurality, or order of winning and losing for STV.

3. System Architecture

3.1 Architectural Design

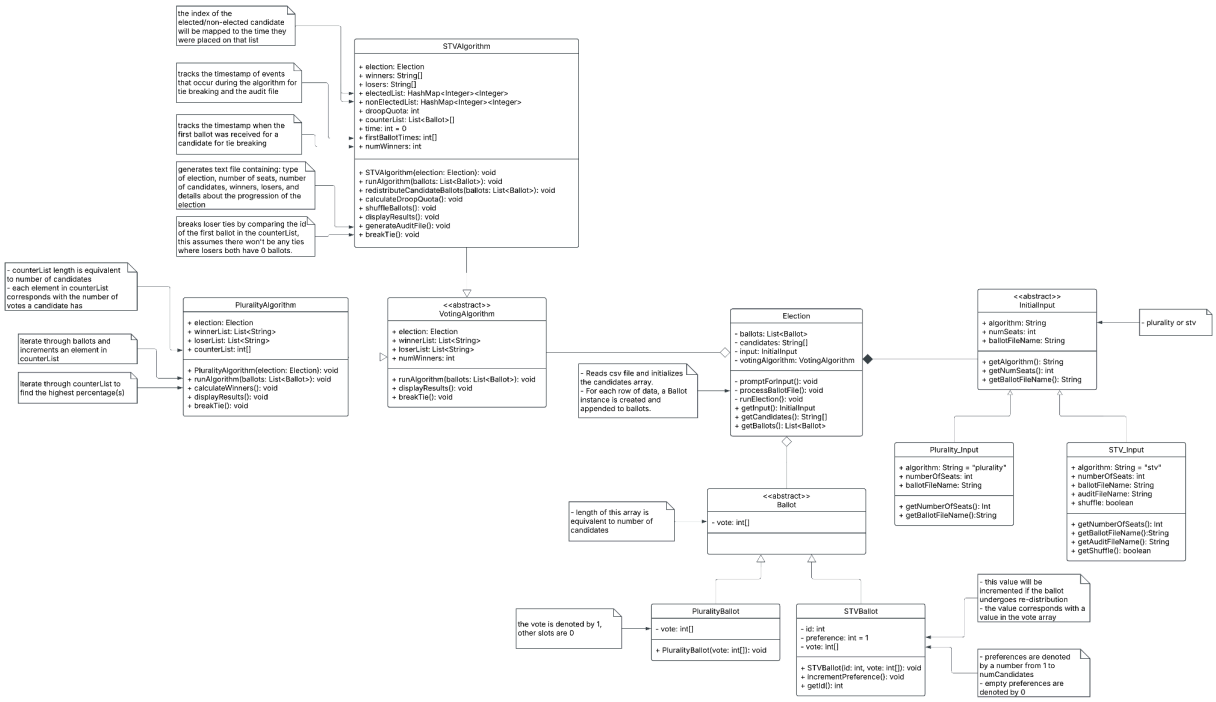

Figure 1 UML Class Diagram (see Appendices 1)

Figure 1 represents the interaction and properties of the different classes that make up the whole system. The main classes are Election, InitialInput, Ballot, and VotingAlgorithm. All of the classes except for Election are abstract classes and are broken down into more specific classes to account for the differences between plurality and STV. An election contains an InitialInput, a VotingAlgorithm, and Ballots. VotingAlgorithm has a reference to the Election object because information is taken from it to complete the algorithm successfully.

3.2 Decomposition Description

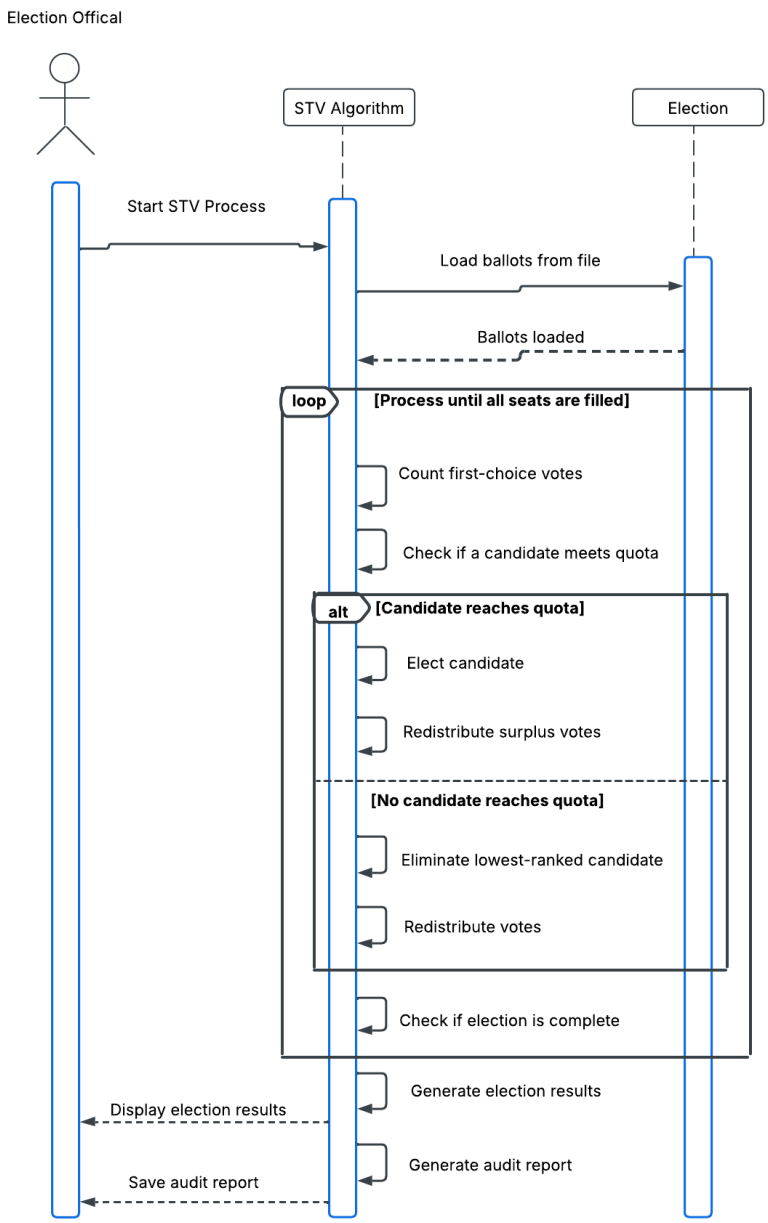

Figure 2 Sequence Diagram for STV (see Appendices 2)

The sequence diagram (figure 2) represents the flow of events in the STV election process, beginning with the Election Official initiating the election. The System then loads ballots from a file in the election class, which contains the voters' preferences. The election proceeds in a loop where votes are counted based on first-choice preferences, continuing until all seats are filled. During this process, candidates who meet the quota (a predefined threshold of votes) are

elected, and their surplus votes are redistributed to the next preferred candidates on each ballot. If no candidate meets the quota, the lowest-ranked candidate is eliminated, and their votes are redistributed to the next preferred candidates. This iterative process continues until all seats are filled. Once the election is complete, the system generates the results and an audit file, which are displayed to the Election Official and saved for official records.

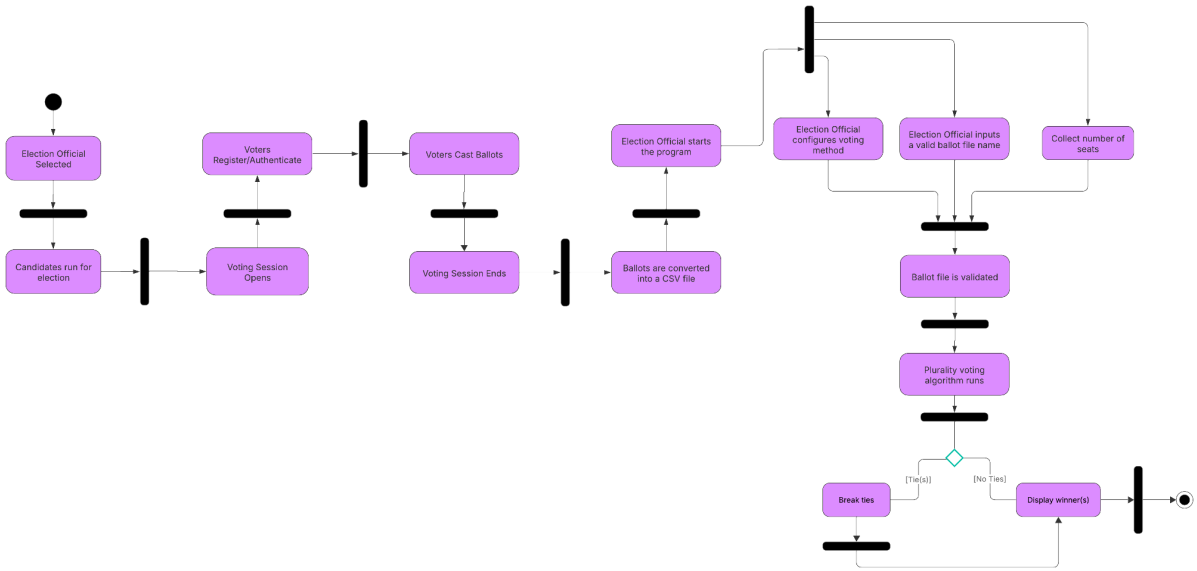

Figure 3 UML Activity Diagram for Plurality election (see Appendices 3)

Plurality voting is one of the two that the system will work with. Figure 3 represents the activities involved with the plurality election. In this diagram, it shows all major activities that are required to happen in order for the system to execute plurality voting. Election officials are required to supervise the election process. Candidates need to run for the election in order for voters to start voting. Once all ballots are cast and the voting session ends, the ballots are converted into a CSV file and the election official will run the voting system. Inputs for file name and number of seats are required before the ballot file is validated and the plurality voting algorithm starts running. If there is a clear winner with the most votes, the winner(s) are displayed. In the case of a tie, the tie will be broken and then display results.

3.3 Design Rationale

Object-oriented architecture was selected due to the organization, reusability, and maintainability of the code base. Single file architecture was considered, but not chosen due to the more time it would take to add new features and fix bugs. Object-oriented could be challenging with many classes, methods, and objects, however in the end, it was chosen as smaller classes were more manageable and easier to understand. Adding new classes would also be easier and will have minimal impact on the existing code.

4. DATA DESIGN

4.1 Data Description

The voting system consists of data related to the candidates, ballots, and election results. The data will be inputted via a CSV file and terminal prompt input. It will be processed and will give results based on the different voting systems that were selected.

- **Candidates**: The system will store candidates' names in a string array, allowing for easy access and tracking for generating audit reports.
- **Ballots/Counting**: The ballots are read from a CSV file, and depending on the election type that was picked, the ballots are stored differently.
    - **For plurality**, the ballots are simply stored as an integer array with the same length as the candidate name array, and each index of the ballot array represents the vote for the candidate in that array.
    - **For STV**, the ballots are stored in the same way, but have an additional property to track the current preference number of the ballot. If a candidate wasn't ranked, then the preference for that candidate would be represented by 0.
- **Election Configuration**: The system will store election-related information such as the number of seats to be filled, the election algorithm (plurality or STV), and the file containing the ballots. This information will be stored in variables and passed as parameters during the election process.
- **Droop Quota Calculation**: For STV elections, the Droop quota will be calculated using the formula:
    - $\left\lfloor \left(\frac{numberOfBallots}{numberOfSeats+1} \right) \right\rfloor + 1$
    - This value will be computed dynamically based on the input data for the election.
- **Audit Log**: A log file will only be generated for STV elections, and it is to report:
    - **Number of seats** available
    - **Number of candidates**
    - **List of winners**
    - **List of losers**
    - **List of elected candidates** (including the order they were elected in STV)
    - **List of non-elected candidates** (including the order they were eliminated in STV)

The program will process the data from the CSV file and transform it into these data structures for further manipulation. The data structures will allow efficient processing during the election, and will be used to generate results, and logs.

4.2 Data Dictionary

Name	Type	Description
Audit log	File	The audit file produced after STV election
Ballot	Array object	Contain the votes for all candidates within the same index
Candidates	String Array	List containing all the candidates
Droop Quota	Integer	The calculated quota that determined the elected candidate for the STV election
List of elected candidates	HashMap<Integer><Integer>	Candidates mapped by their index to the time they were put on the elected list.
List of losers	String Array	List of losers for STV
List of non-elected candidates	HashMap<Integer><Integer>	Candidates mapped by their index to the time they were put on the non-elected list.
List of winners	String Array	List of winners for STV
Number of candidates	Integer	Number of candidates participating
Number of seats	Integer	Number of seats available

5. Component Design

This section details the core components of the voting system, outlining the procedural flow for STV sequence diagram and the activity diagram for plurality voting.

1. STV Process

- Start STV process
- Loading ballots from the election (where it contains the ballot information)
- Counting votes in rounds until all seats are filled
- Handling surplus votes redistribution if candidates reached quota
- Eliminating lowest-ranked candidates if no candidates reached quota
- Checking if all seats are filled and election is complete
- Generating results and audit file

2. Plurality Process

- Election official is selected
- Candidates run for election
- Voting session opens
- Voters register
- Voters cast ballots
- Voting session ends
- Ballots are converted into CSV file
- Election official starts program
- Election official configures voting method
- Election official inputs a valid ballot file name
- Election official inputs numbers of seats
- Ballot file is validated by the system
- Plurality voting algorithm runs
- If tie: break ties
- Display winner(s)

6. Human Interface Design

6.1 Overview of User Interface

The user will be using the terminal to both input the ballot CSV file, the number of seats, election type, and also the ballot file name if STV election were picked.

6.2 Screen Images

```
  C:\User\Username\...\Voting_System> java VotingSystem
  Select voting algorithm: [Plurality or STV]
      If Plurality:
  Input in this format: [file_path] [number_of_seats]
      If STV:
  Input in this format: [file_path] [number_of_seats] [audit_file_name]
[toggle on/off]

  Input:
```

6.3 Screen Objects and Actions

The user will run the program inside of the terminal. When the program starts, it will print onto the terminal the order in which the user has to input the needed information depending on which algorithm is selected.

7. Requirements Matrix

Requirement ID	Requirement Description	Design Component
UC_000	Select Voting Algorithm	InitialInput
UC_001	Initial Input	InitialInput
UC_002	Read CSV	Election
UC_003	Process STV voting	STVAlgorithm
UC_004	Process Plurality Voting	PluralityAlgorithm
UC_005	Create Audit File	STVAlgorithm
UC_006	Turn off Shuffle	InitialInput
UC_007	Handling Tie	VotingAlgorithm

8. Appendices

1. UML Class Diagram
2. Sequence Diagram for STV
3. UML Activity Diagram for Plurality Election